Trabajo Práctico N.º 2

Diseño de Clases

Indicaciones:

- ***i.*** *Al comenzar el TP de debe clonar el repositorio correspondiente y crear la rama local con su nombre.*
- ***ii.*** *Para cada ejercicio se debe crear un Proyecto con el nombre en mayúsculas TP2_EJX.*
- ***iii.*** *Para versionar el trabajo local se debe usar "commit" y para enviar al repositorio remoto usar "push"*
- ***iv.*** *En el método "static void main()" se deben crear las instancias de las clases diseñadas para invocar sus métodos y comprobar su funcionamiento.*
- ***v.*** *En todas las clases, de definen los Constructores, los métodos Getters() y Setters() y los necesarios según corresponda*

Ejercicio 1. Implementar la clase **Fecha** con atributos para el mes, el día y el año. Cada objeto de esta clase representa una fecha, que almacena el día, mes y año como enteros. Se deben incluir los siguientes métodos:

Fecha
- dia: int - mes: int - año: int
+ Fecha() + Fecha(d: int, m: int, a: int) + getDia(): int + getMes(): int + getAño(): int + setDia(d: int): void + setMes(m: int): void + setAño(a: int): void + reiniciar(d: int, m: int, a: int): void + adelantar(d: int): void + imprimir(): void - valida(): boolean - normalizar(): void - dias(mes: int): int + diferenciaEnDias(f: Fecha): int + comparar(f: Fecha): int + clonar(): Fecha + toString(): String + formatear(fmt: String): String + static esBisiesto(a: int): boolean + static hoy(): Fecha

- ☐ **Constructor** por defecto y **Constructor** con parámetros
- ☐ Métodos de acceso (**getters** y **setters**)
- ☐ **reiniciar(int d, int m, int a)** para reiniciar la fecha de un objeto existente
- ☐ **adelantar(int d)** para avanzar una cantidad de días
- ☐ **imprimir()** para mostrar la fecha
- ☐ **valida()** que determine si es una fecha correcta
- ☐ **normalizar()** que corrija los valores para que estén en rangos correctos
- ☐ **dias(int mes)** que devuelve el número de días de cada mes
- ☐ Método estático **esBisiesto(int año)** que determine si un año es bisiesto

Agregar también los siguientes métodos adicionales:

- ☐ **toString()** para representación en formato DD/MM/AAAA
- ☐ **clonar()** para crear una copia de la fecha
- ☐ **comparar(Fecha otra)** para comparar fechas
- ☐ **diferenciaEnDias(Fecha otra)** para calcular días entre fechas
- ☐ **formatear(String formato)** para personalizar la presentación
- ☐ **estático hoy()** que devuelve la fecha actual

Nota: un año es bisiesto si es divisible por 400, o si es divisible por 4 pero no por 100. Por ejemplo, los años 1992 y 2000 son años bisiestos mientras que 1997 y 1800 no son bisiestos.

En el método **main**, instanciar objetos de la clase Fecha para calcular la edad de una persona, comparar dos fechas, calcular los días restantes para un eventos y otros ejemplos de uso.

Ejercicio 2. Desarrollar una clase llamada **Complejo** que permita representar y operar con números complejos. La clase debe tener atributos para la parte real e imaginaria e implementar las operaciones básicas (suma, resta, multiplicación y división) como:

- Métodos de **instancia** que modifican el objeto actual.
- Métodos **estáticos** que reciben dos objetos Complejo y devuelven un nuevo objeto Complejo.

$c \leftarrow c + a$	$c \leftarrow a + b$
$c \leftarrow c - a$	$c \leftarrow a - b$
$c \leftarrow c \times a$	$c \leftarrow a \times b$
$c \leftarrow c \div a$	$c \leftarrow a \div b$
$\lvert z \rvert$	
$\overline{z}$	

Complejo
- real: double - imaginaria: double - static final double EPSILON: double
+ Complejo() + Complejo(double real, double imaginaria) + getReal(): double + getImaginaria(): double + setReal(double real): void + setImaginaria(double imaginaria): void + sumar(Complejo otro):void + restar(Complejo otro):void + multiplicar(Complejo otro):void + dividir(Complejo otro):void + modulo(): double + argumento(): double + conjugado(): Complejo + equals(Object obj): boolean + toString(): String + static sumar(Complejo c1, Complejo c2): Complejo + static restar(Complejo c1, Complejo c2): Complejo + static multiplicar(Complejo c1, Complejo c2): Complejo + static dividir(Complejo c1, Complejo c2): Complejo

Deben estar presentes los métodos constructores así como los de acceso a los atributos.

Incluir métodos útiles adicionales como:

- **modulo()**: calcula $|z| = \sqrt{(a^2 + b^2)}$
- **argumento()**: calcula $\theta = \arctan(b/a)$
- **conjugado()**: retorna el conjugado (a - bi)
- **opuesto():** retorna el opuesto (-a - bi)
- **potencia(int n)**: calcula z^n
- **equals(Object otro)**: compara igualdad con tolerancia
- **toString()**: representación en formato a+bi

En el método **main**, instanciar varios objetos de la clase Complejo y probar su funcionamiento.

Ejercicio 3. Desarrolle una implementación en Java de la clase **Racional** que permita representar y operar con números racionales (fracciones). La implementación debe cumplir con los siguientes requisitos:

- La clase debe tener atributos para almacenar el numerador y denominador y debe implementar operaciones básicas como **métodos de instancia**: Suma, Resta, Multiplicación y División.

- Las mismas operaciones se debe proporcionar como **métodos estáticos** que reciben dos objetos Racional y retornan un nuevo objeto con el resultado.

Incluir métodos útiles para el manejo de números racionales en aplicaciones prácticas:

- Simplificación de fracciones
- Conversión a decimal
- Comparación de racionales
- Representación como cadena de texto

En el método **main**, instanciar varios objetos de la clase Racional y probar su funcionamiento.

Racional
- numerador: int - denominador: int
+ Racional() + Racional(int numerador, int denominador) + getNumerador(): int + getDenominador(): int + setNumerador(int numerador): void + setDenominador(int denominador): void + sumar(Racional r):void + restar(Racional r):void + multiplicar(Racional r):void + dividir(Racional r):void + simplificar(): Racional + aDecimal(): double + toString(): String + equals(Object obj): boolean + compareTo(Racional r): int + static sumar(Racional r1, Racional r2): Racional + static restar(Racional r1, Racional r2): Racional + static multiplicar(Racional r1, Racional r2): Racional + static dividir(Racional r1, Racional r2): Racional - static mcd(int a, int b): int

Ejercicio 4. Codifica una clase **Polinomio** que sirva para el manejo de polinomios matemáticos con las siguientes especificaciones:

- Un atributo **grado** de tipo entero
- Un arreglo **coeficientes** de tipo double para almacenar los coeficientes donde el índice representa la potencia de x

Proporcionar los siguientes métodos:

- Constructor que recibe un arreglo de coeficientes
- Constructor que recibe un grado y crea un polinomio con todos los coeficientes en cero
- Métodos de **instancia** para operaciones básicas que modifican los atributos del Polinomio:
  - ➔ **sumar(Polinomio p)**: Suma dos polinomios
  - ➔ **restar(Polinomio p)**: Resta un polinomio del actual
  - ➔ **multiplicar(Polinomio p)**: Multiplica dos polinomios
  - ➔ **dividir(Polinomio p)**: Divide el polinomio actual por otro y devuelve el cociente y el residuo como un arreglo

- Métodos **estáticos** que realizan las operaciones y devuelven el resultado en otro objeto:
  - **sumar(Polinomio p1, Polinomio p2)**
  - **restar(Polinomio p1, Polinomio p2)**
  - **multiplicar(Polinomio p1, Polinomio p2)**
  - **dividir(Polinomio p1, Polinomio p2)**
- Métodos adicionales útiles:
  - **evaluar(double x)**: Evalúa el polinomio en un punto usando el algoritmo de Horner
  - **derivar()**: Calcula la derivada del polinomio
  - **integrar()**: Calcula la integral indefinida del polinomio
  - **calcularRaices()**: Calcula las raíces para polinomios de grado ≤ 2
  - **toString()**: Convierte el polinomio a una representación de cadena en formato matemático

Polinomio
- grado: int - coeficientes: double[]
+ Polinomio(coeficientes: double[]) + Polinomio(grado: int) + getGrado(): int + getCoeficientes(): double[] + getCoeficiente(i: int): double + setCoeficiente(i: int, c: double) + ingresarPolinomio(): void + agregarCoeficiente(coef: double, exp: int): void + sumar(p: Polinomio):void + restar(p: Polinomio):void + multiplicar(p: Polinomio):void + dividir(p: Polinomio): Polinomio[] + evaluar(x: double): double + derivar(): Polinomio + integrar(): Polinomio + calcularRaices(): double[] + toString(): String + static sumar(p1: Polinomio, p2: Polinomio): Polinomio + static restar(p1: Polinomio, p2: Polinomio): Polinomio + static multiplicar(p1: Polinomio, p2: Polinomio): Polinomio + static dividir(p1: Polinomio, p2: Polinomio): Polinomio[]

Ejercicio 5. Codifica una clase Conjunto que sirva para el manejo de conjuntos matemáticos con las siguientes métodos:

a) Contructores

b) Getters & Setters

c) Unión

d) Intersección

e) Diferencia

f) Simetría

<u>Observaciones.</u> Cada ejercicio debe proponer código en el **método main** de la clase principal para:

- Instanciar objetos
- Probar los getters & setters
- Probar toString() y equals
- Invocar operaciones (métodos) de instancia y de clase, que involucre 1 objeto y/o 2 objetos

Ejercicio 6. Una empresa dedicada a la venta de Jugos, desea registrar la información de sus vendedores: *legajo*, *zonaDeVenta*, *ventasMensuales*, *totalAnual* y *comisión*.

Las *ventasMensuales* representan las ventas realizadas por el vendedor en cada uno de los doce meses del año, *totalAnual* representa la suma de las ventas en un año y *comisión* un porcentaje que depende del totalAnual. Los *datos* personales del vendedor se contienen en una instancia de una clase Persona con los siguientes atributos: *apellidos*, *nombres* y *dni*.

A) Confecciona el diagrama UML con las clases necesarias que permitan responder los siguientes requerimientos:

B) Crea una instancia de la clase Vendedor y solicita al usuario los datos para inicializar sus atributos, excepto vtasMensuales, y comisión que se inicializan con ceros.

C) Generar, en forma aleatoria, cada monto vendido en un mes por el vendedor y asignarlo en el atributo correspondiente.

D) Calcular el totalAnual conseguido por un vendedor.

E) Determinar la comisión del vendedor, teniendo en cuenta que: no habrá comisión si el totalAnual no alcanza los $ 50.000 como mínimo, será del 15% si el totalAnual se encuentra entre $50.000 y $75.000, será del 20% si el totalAnual se encuentra entre $75.000 y $100.000, será del 30% si el totalAnual supera los $100.000.

F) Mostrar por salida de consola la comisiones de al menos 3 vendedores.

Ejercicio 7. Desarrollar una aplicación para la asignación de pacientes a los médicos de un hospital en el servicio de urgencias.

Un **médico** representa al especialista que está en urgencias para atender a los pacientes. Un médico se caracteriza por:

- Una cadena de texto que representa el nombre y un entero para el número de matrícula profesional. Estos valores se establecen en el constructor y no pueden cambiar.
- Especialidad (familia, traumatología, pediatría, internista u otorrino). La especialidad del médico se puede establecer en el constructor y no podrá cambiar. Si no se indica explícitamente se entiende que la especialidad es “familia”.
- Cupo. Entero que representa el número máximo de pacientes que el médico puede atender. Este valor se puede establecer en el momento de la construcción. En caso contrario se entiende que el cupo es de 10. El cupo de un médico puede cambiar por necesidades del servicio.
- Lista de las historias de los pacientes que está atendiendo (el concepto historia se describe más adelante).
- Disponibilidad: indica si aún puede atender pacientes (el número de historias es inferior al cupo máximo).

Una **historia** representa la información del paciente que es atendido en el servicio de urgencias. La historia se caracteriza por:

- Un identificador. Es un entero que se asigna automáticamente, y de forma consecutiva, al crear la historia. Una vez establecido no puede cambiar. El primer identificador es el 1.
- Código de paciente (cadena de caracteres). Esta propiedad no cambia una vez establecida.
- Estado inicial del paciente que puede ser: grave, medio o leve.
- Especialidad del médico que debe atender al paciente.
- Médico asignado. Esta información no se conoce hasta que se asigna la historia a uno de los médicos disponibles. Sólo se puede establecer una vez y una vez establecido no puede cambiar.

- Alta paciente. Indica si el paciente asociado a la historia ha recibido el alta médica. Inicialmente cuando se crea una historia esta propiedad tiene el valor falso.

El estado inicial y la especialidad se establecen en el constructor y no pueden cambiar. Todos los atributos se pueden consultar.

La funcionalidad principal de la aplicación es *asignar una historia a un médico* y *dar el alta médica al paciente asociado a una historia*. Ambas acciones se describen a continuación:

- Las historias de los pacientes se asignan a los médicos en función de la especialidad y la disponibilidad. De manera general, para **asignar una historia a un médico** se tiene que cumplir que la especialidad establecida en la historia coincida con la especialidad del médico y que éste tenga disponibilidad para atenderlo. Si se cumplen estas condiciones se añade la historia a la lista del médico y se establece el médico como médico de la historia. La ejecución del método debe devolver un valor booleano que indique si la asignación se ha realizado con éxito o no.
- **Dar el alta médica al paciente de una historia**. Esta acción sólo se puede realizar si la historia tiene un médico asignado e implica establecer a verdadero la propiedad alta paciente y eliminar la historia de la lista de historias del médico que la estaba atendiendo. Nótese que una historia que cuyo paciente ha recibido el alta médica debe mantener la asociación con el médico que la atendió. Por último, esta acción no puede ser revertida, esto es, la propiedad alta paciente no podrá volver a tener el valor falso.

Implementa la clase que representa el **cuadro médico** de urgencias. El cuadro médico lo constituye la lista de médicos que están trabajando en urgencias en un momento dado. Para facilitar las búsquedas de médicos disponibles, la lista debe estar organizada por especialidades y, los médicos de cada especialidad deben estar ordenados de menor a mayor disponibilidad. La funcionalidad que proporciona esta clase es:

- Insertar y eliminar médico.
- Obtener el médico de mayor disponibilidad para una especialidad dada. En el caso de que no haya ningún médico disponible, o no exista la especialidad indicada en el cuadro médico, se devolverá null.

Implementa el siguiente programa para probar la funcionalidad:

1. Crea los siguientes médicos:
   - nombre “Nortes”, número de matrícula 5677 y especialidad pediatría.
   - nombre “Gutiérrez”, número de matrícula 4568 y especialidad familia.
   - nombre “Mayor”, número de matrícula 7890, especialidad familia.
   - nombre “Guillén”, número de matrícula 4567 y especialidad traumatología.
   - nombre “Mateo”, número de matrícula 8976, especialidad traumatología.
2. Crea una variable que haga referencia a un objeto de tipo cuadro médico e inserta todos los médicos anteriores.
3. Crea una lista de historias e inserta las historias de los siguientes pacientes:
   - Rocío, en estado leve, debe ser atendida por un pediatra.
   - Andrés, en estado grave, debe ser atendido por un traumatólogo.
   - Juan, en estado medio, debe ser atendido por un traumatólogo.
   - Marieta, en estado leve, debe ser atendida por un médico de familia.
   - Alfonso, en estado medio, debe ser atendido por un médico de familia.
4. Recorre la lista de historias y asigna a cada paciente el médico disponible según la consulta del cuadro médico. Imprime las asignaciones establecidas:

   nombre del médico – nombre del paciente – número de historia